

Just-in-Time-Privilege-Elevation

Ein Modul für zeitlich befristete,
zwei-faktor-gesicherte Rechte-Erhöhung

Autor: Lukas Cvitanović
Matrikelnr.: 8051900
Kurs: WDS125

30. Juni 2026

Quellcode:
<https://github.com/DrLuggels/Methoden-und-Werkzeuge-der-Softwareentwicklung->
Abgabe-Tag: v1.0

Inhaltsverzeichnis

1	Einleitung und Motivation	2
2	Anforderungen	2
3	Stand der Technik	2
4	Konzept und Architektur	3
4.1	Einordnung in die Bestandsarchitektur	3
4.2	Lebenszyklus, Ablauf und Datenmodell	3
5	Sicherheitsanalyse	3
5.1	Bedrohungsklassen	4
5.2	Vergleich mit dem Grundsystem	4
5.3	Quantitativer Angriffsaufwand	4
5.4	Fail-closed als durchgängiges Prinzip	4
6	Implementierung	5
6.1	Modul-Struktur	5
6.2	Die Schnittstellen (Ports)	5
6.3	Der sicherheitskritische Pfad: Confirm	5
6.4	TOTP nach RFC 6238	5
7	Evaluation	6
7.1	Tests und Abdeckung	6
7.2	Durch Tests aufgedeckte Defekte	6
7.3	Limitierungen	6
8	Fazit und Ausblick	6

1 Einleitung und Motivation

Administrative Berechtigungen werden in vielen Systemen dauerhaft vergeben: Ein Konto besitzt erhöhte Rechte unabhängig davon, ob diese im Moment benötigt werden. Das widerspricht dem Least-Privilege-Prinzip [1], vergrößert den Schaden einer Kontoübernahme, da ein kompromittiertes Administratorkonto jederzeit voll handlungsfähig ist, und erschwert die Nachvollziehbarkeit. Als Antwort hat sich *Just-in-Time*-Zugriff (JIT) etabliert: Rechte werden nur für den konkreten Bedarfsmoment gewährt und laufen automatisch ab. Der Zeitraum für möglichen Missbrauch sinkt auf wenige Minuten, und jede Erhöhung wird zu einem protokollierbaren Ereignis.

Die vorliegende Arbeit entwirft und implementiert ein eigenständiges, wiederverwendbares Modul, das JIT-Privilege-Elevation als Bibliothek bereitstellt, vom Trägersystem entkoppelt ist und beispielhaft in eine bestehende PaaS-Umgebung integriert wird. Es folgen Anforderungen (Abschnitt 2), Stand der Technik (3), Konzept (4), Sicherheitsanalyse nach STRIDE (5), Implementierung (6) und Evaluation (7).

2 Anforderungen

Funktional: Ein Benutzer kann eine zeitlich befristete Rechte-Erhöhung *beantragen*, die erst nach erneuter Zwei-Faktor-Bestätigung (Step-Up-Authentication) wirksam wird. Aktive Erhöhungen lassen sich vorzeitig widerrufen und laufen andernfalls automatisch ab; jeder Schritt wird manipulationssicher protokolliert.

Nicht-funktional: *Fail-closed*, das heißt im Zweifel oder bei jedem Fehler wird kein Recht gewährt; *Unabhängigkeit* vom Trägersystem (Ports-and-Adapters); *Nachvollziehbarkeit* durch einen lückenlosen, integritätsgesicherten Audit-Trail; *Korrektheit*, verifiziert durch automatisierte Tests inklusive Nebenläufigkeit.

3 Stand der Technik

Verfahren zur kontrollierten Rechteerhöhung sind etabliert und unterscheiden sich vor allem in Granularität und Betriebsaufwand (Tabelle 1). Während Vault [2], Boundary [3] bzw. Teleport [4] und OIDC-Provider wie Keycloak [5] eigene Infrastruktur bzw. hohen Betriebsaufwand voraussetzen, verfolgt das hier entwickelte Modul einen leichtgewichtigen Ansatz: Es führt die Erhöhung als eingebettete Bibliothek statt als externe Infrastruktur durch und bleibt durch das Ports-and-Adapters-Muster [6] unabhängig vom Trägersystem.

Tabelle 1: Einordnung verwandter Systeme

System	Ansatz	Abgrenzung
HashiCorp Vault (SSH)	kurzlebige SSH-Zertifikate	OS-Ebene, eigene Infrastruktur
Boundary / Teleport	Session-Broker mit Recording	mächtig, hoher Betriebsaufwand
Keycloak / Authentik	OIDC-basierte Re-Auth	Identity-Provider-zentriert
Dieses Modul	eingebettete JIT-Erhöhung	leichtgewichtig, integrierbar

4 Konzept und Architektur

4.1 Einordnung in die Bestandsarchitektur

Abbildung 1 zeigt die Integration in eine typische DEV/PROD-Topologie. Grau dargestellt ist der Status quo: zwei gleichwertige, vollständige Stacks (Entwicklung und Produktion) aus jeweils Frontend, Backend und Datenbank sowie der Deploy-Pfad über das Git-Repository. Farbig hervorgehoben ist die Ergänzung durch das vorgestellte Modul: Der Benutzer beantragt beim JITPE-Dienst eine zeitlich befristete Erhöhung und bestätigt sie per Zweitfaktor. *Beide* Umgebungen behandeln JITPE identisch – vor jeder privilegierten Aktion fragt das jeweilige Backend dort nach (*verify*); nur bei aktiver Erhöhung wird die Aktion ausgeführt, andernfalls verweigert (fail-closed). Intern folgt das Modul dem Ports-and-Adapters-Muster: Der Kern kennt das Host-System nicht, sondern spricht ausschließlich gegen Interfaces (siehe Abschnitt 6).

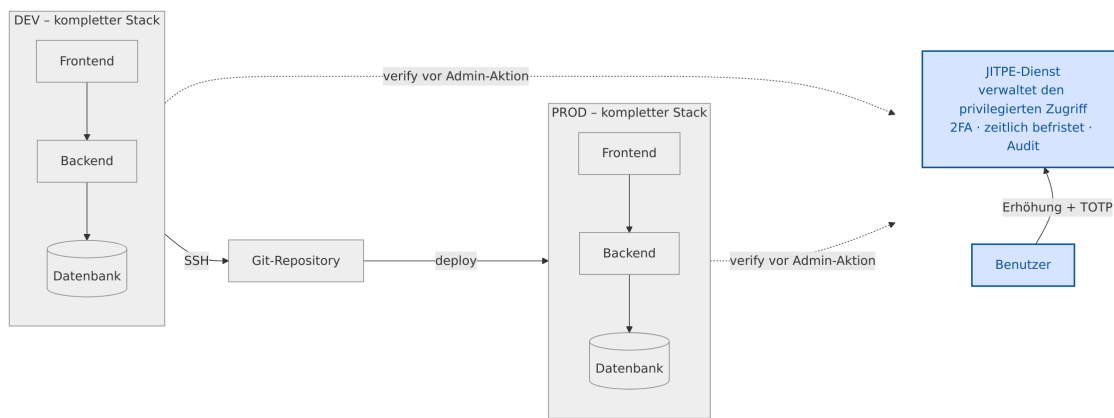


Abbildung 1: Integration: DEV und PROD (Status quo, grau) sind gleichwertige Stacks; beide fragen das JITPE-Modul (blau) identisch vor jeder privilegierten Aktion (*verify*)

4.2 Lebenszyklus, Ablauf und Datenmodell

Eine Rechte-Erhöhung (Grant) durchläuft die Zustände **pending** (Antrag gestellt), **active** (per TOTP bestätigt) und schließlich einen der terminalen Zustände **expired**, **revoked** oder **denied**. Alle Endzustände sind terminal; ein abgelaufener, widerrufen oder abgelehnter Grant wird nie wieder aktiv (fail-closed). Der Ablauf umfasst damit vier Phasen: Antrag, Bestätigung per TOTP (Step-Up; ohne gültigen zweiten Faktor keine aktive Erhöhung), Nutzung der geschützten Aktion und automatischer Ablauf durch einen Hintergrund-Ticker. Persistiert wird dies in drei Tabellen: den Grants, dem Audit-Log und einer Tabelle der bereits verbrauchten TOTP-Zeitfenster für den benutzerweiten Replay-Schutz. Die Audit-Tabelle bildet eine Hash-Kette: Jeder Eintrag enthält den SHA-256-Hash [7] seines Vorgängers, sodass jede nachträgliche Manipulation die Kette am Folgeeintrag bricht.

5 Sicherheitsanalyse

Die Analyse folgt STRIDE [8] und ordnet jeder Bedrohung eine konkrete Gegenmaßnahme im Entwurf bzw. im Code zu. Annahmen: Der Host authentifiziert die Primäridentität, TLS sichert die Transportwege (Browser/Backend und Backend/Datenbank), und der TOTP-Seed liegt verschlüsselt im Secret-Store und verlässt ihn nie im Klartext.

5.1 Bedrohungsklassen

Jede Klasse ist im Code verortet: *Spoofing* scheitert am TOTP-Step-Up; *Tampering* an der SHA-256-Hash-Kette (`audit.go`); *Repudiation* am lückenlosen, pflichtigen Protokoll; *Information Disclosure* an verschlüsselten Seeds (AES-256-GCM [9]) und 128-Bit-Zufalls-IDs. Die Kernklasse *Elevation of Privilege* wird mehrfach abgesichert: Die `PolicyEngine` deckelt Rolle und Dauer, ein Ticker erzwingt den Ablauf (`ExpireDue`), die Aktivierung ist atomar gegen Doppel-Erhöhung, und die Autorisierung erfolgt serverseitig und fail-closed (`Check`). Einzig *Denial of Service* ist kein Gewinn (siehe unten).

5.2 Vergleich mit dem Grundsystem

Tabelle 2 stellt die Klassen dem Grundsystem gegenüber, dem üblichen Zugriff über dauerhafte SSH-Schlüssel und stehendes `sudo`. In fünf der sechs Klassen ist das Modul überlegen; nur bei der Verfügbarkeit (D) ist fail-closed ein bewusster Trade-off zugunsten der Sicherheit. Kernunterschied: Privileg ist im Grundsystem der Dauerzustand, im Modul die befristete, geprüfte Ausnahme.

Tabelle 2: JIT-Modul gegenüber dem Grundsystem (dauerhafte Schlüssel + sudo)

Klasse	Grundsystem	JIT-Modul
Spoofing	ein Schlüssel, unbefristet gültig	zwei Faktoren, 30-s-TOTP mit Replay-Schutz
Tampering	Log lokal und veränderbar	zentrale, verkettete Hash-Sicherung
Repudiation	Login ohne Begründung	Akteur + Pflicht-Begründung, unveränderlich
Info Disclosure	Key-Leak = Dauerzugriff	kurzlebiger Grant, kein Langzeit-geheimnis
Denial of Service	bei Ausfall verfügbar	fail-closed (bewusster Trade-off)
Elevation	Privileg ist Dauerzustand	Privileg ist befristete, geprüfte Ausnahme

5.3 Quantitativer Angriffsaufwand

Der Vorteil lässt sich beziffern (Tabelle 3). Der Rechenweg ist für alle Brute-Force-Werte identisch:

$$t = \frac{\text{Suchraum}}{(\text{Versuche/Sekunde}) \cdot 3,15 \cdot 10^7 \text{ s/Jahr}}.$$

Für den verschlüsselten TOTP-Seed (AES-256) etwa ergibt sich bei optimistischen 10^{18} Versuchen/s: $t = 2^{256} / (10^{18} \cdot 3,15 \cdot 10^7) \approx 3,7 \cdot 10^{51}$ Jahre. Der entscheidende Punkt ist nicht nur die Größe der Räume, sondern die *Rotation*: Da der TOTP-Code alle 30 s wechselt und je Antrag nur ein Versuch möglich ist (ein falscher Code verwirft den Antrag), akkumuliert ein Angreifer keinen Fortschritt – anders als beim statischen Geheimnis des Grundsystems.

5.4 Fail-closed als durchgängiges Prinzip

Jeder Fehlerpfad, sei es ein Policy-Fehler, fehlender oder falscher TOTP, Replay, abgelaufene Frist oder Store-Fehler, führt zu keinem aktiven Grant. `Check` liefert im Zweifel `false`. Dies ist durch Tests belegt (u. a. `TestCheck_FailsClosedOnStoreError`).

Tabelle 3: Aufwand für dauerhaften Admin-Zugang (Annahme: Primitive sicher)

Angriff	Grundsystem	JIT-Modul
Dauer-Geheimnis knacken	1 statischer Hash, endlich (Sek. bis Tage)	AES-256-Seed: $2^{256} \approx 10^{51}$ Jahre
ID/Token erraten	—	$2^{128} \approx 1,1 \cdot 10^{22}$ Jahre
Code online raten	statisch, kumulativ	10^6 , nur 1 Versuch je Antrag
Lebensdauer gestohlenen Geheimnis	unbegrenzt	≤ 30 min (dann <code>expires_at</code>)

6 Implementierung

6.1 Modul-Struktur

Das Modul ist in Go geschrieben und besteht aus dem Kern (Domänentypen, State-Machine, die Schnittstellen in `ports.go`, die Hash-Kette in `audit.go`, eine Beispiel-Policy sowie die zentrale Logik in `module.go`) und drei Adapttern: einem In-Memory-Store, einem dateibasierten Store (JSON-Persistenz, atomar per Temp-Datei und Rename) und einem TOTP-Verifier nach RFC 6238. Der Kern hat keine externen Abhängigkeiten.

6.2 Die Schnittstellen (Ports)

Der Kern definiert fünf Schnittstellen, die das Host-System implementiert. Die zentrale ist **Store** für die Persistenz: Sie umfasst das Anlegen und Lesen eines Grants, die Zustandsübergänge (Aktivieren, Ablehnen, Widerrufen), die Abfrage aktiver Grants sowie zwei sicherheitskritische Operationen: **ExpireDue**, das fällige Grants als abgelaufen markiert, und **MarkTOTPUsed**, das ein TOTP-Zeitfenster atomar als verbraucht vermerkt (Replay-Schutz). Die übrigen vier Schnittstellen kapseln den Zweitfaktor (**TOTPVerifier**), das Protokoll (**AuditSink**), die Freigaberegeln (**PolicyEngine**) und die austauschbare Zeitquelle (**Clock**).

6.3 Der sicherheitskritische Pfad: Confirm

Die Bestätigung ist der Step-Up-Schritt und durchläuft drei Stufen, von denen jeder Fehlschlag die Erhöhung verhindert. Zuerst wird der Zweitfaktor geprüft; ein falscher Code verwirft den Antrag. Anschließend markiert der Replay-Schutz das zugehörige Zeitfenster atomar als verbraucht, sodass ein bereits genutzter Code abgewiesen wird. Erst dann wird der Grant atomar von **pending** auf **active** gesetzt, sodass bei zwei parallelen Bestätigungen nur eine gewinnt. Nur wenn alle drei Stufen erfolgreich sind, gilt die Erhöhung.

6.4 TOTP nach RFC 6238

Der Zweitfaktor-Verifier implementiert RFC 6238 (TOTP) [10] auf Basis von RFC 4226 (HOTP) [11] mit der Standardbibliothek. Die kryptografische Primitive (HMAC-SHA1) stammt aus `crypto/hmac`; der Code-Vergleich erfolgt zeitkonstant (`crypto/subtle`) gegen Timing-Angriffe. Die Korrektheit ist gegen die offiziellen Testvektoren beider RFCs nachgewiesen. Produktiv empfiehlt sich eine auditierte Bibliothek (z. B. `pquerna/otp`).

7 Evaluation

7.1 Tests und Abdeckung

15 automatisierte Tests laufen mit aktiviertem Race-Detector (`go test -race`); die Abdeckung beträgt rund 81 % im Kern und 91 % im TOTP-Adapter. Geprüft werden u. a. Happy-Path, falscher Zweitfaktor, Replay, Fristablauf, Widerruf, Policy-Ablehnung, fail-closed bei Store-Fehler, Nebenläufigkeit und die Integrität der Audit-Hash-Kette.

7.2 Durch Tests aufgedeckte Defekte

Der Nebenläufigkeitstest deckte zwei Defekte auf, die im Einzeltest unsichtbar blieben:

1. **Replay-Sabotage:** Bei parallelen Bestätigungen desselben Antrags setzte der Replay-Verlierer den Grant auf `denied`, bevor der Gewinner aktivieren konnte. Behoben, indem ein Replay den Antrag nicht mehr verwirft, sondern als eigenes Sicherheitsereignis (`replay_blocked`) protokolliert wird.
2. **Data-Race im Audit-Sink:** Der Race-Detector belegte, dass `AuditSink.Emit` nebenläufig aufgerufen wird; die Anforderung der Nebenläufigkeitssicherheit wurde am Interface dokumentiert.

7.3 Limitierungen

Bekannte Grenzen sind ein möglicher DoS auf pending-Anträge bei bekannter GrantID (Minderung: Fehlversuchs-Zähler), die fehlende Phishing-Resistenz von TOTP (Minderung: WebAuthn/FIDO2 [12]) sowie das bewusste Vertrauen in die Host-Authentifizierung. Audit-Schreibfehler auf Nebenpfaden werden best-effort behandelt; produktiv koppelt der Store-Adapter Zustandsänderung und Audit in einer Transaktion.

8 Fazit und Ausblick

Die Arbeit hat gezeigt, dass sich Just-in-Time-Privilege-Elevation als schlankes, wiederverwendbares Modul umsetzen lässt, das bewährte Bausteine (TOTP, SHA-256-Hash-Kette) kombiniert, statt eigene Kryptografie zu entwickeln. Der Mehrwert gegenüber dem Standardansatz wurde quantitativ belegt (Abschnitt 5): Wo stehende SSH-Schlüssel Privileg zum Dauerzustand machen, ist es hier eine befristete, per Zweitfaktor bestätigte und lückenlos protokollierte Ausnahme. Offen bleiben Erweiterungen wie befristetes `sudo`, phishing-resistente Faktoren (WebAuthn/FIDO2) und ein MariaDB-Adapter.

Literaturverzeichnis

- [1] OWASP Foundation, “Least Privilege Principle,” OWASP Community. [Online]. Verfügbar: https://owasp.org/www-community/controls/Least_Privilege_Principle (abgerufen am 8. Juni 2026).
- [2] HashiCorp, “Vault – SSH Secrets Engine,” Vault Documentation. [Online]. Verfügbar: <https://developer.hashicorp.com/vault/docs/secrets/ssh> (abgerufen am 8. Juni 2026).
- [3] HashiCorp, “Boundary Documentation.” [Online]. Verfügbar: <https://developer.hashicorp.com/boundary> (abgerufen am 8. Juni 2026).
- [4] Teleport, “Teleport Documentation.” [Online]. Verfügbar: <https://goteleport.com/docs/> (abgerufen am 8. Juni 2026).
- [5] Keycloak, “Keycloak Documentation.” [Online]. Verfügbar: <https://www.keycloak.org/documentation> (abgerufen am 8. Juni 2026).
- [6] “Hexagonale Architektur,” Wikipedia, die freie Enzyklopädie. [Online]. Verfügbar: https://de.wikipedia.org/wiki/Hexagonale_Architektur (abgerufen am 8. Juni 2026).
- [7] National Institute of Standards and Technology, “FIPS 180-4: Secure Hash Standard (SHS),” NIST CSRC. [Online]. Verfügbar: <https://csrc.nist.gov/pubs/fips/180-4/upd1/final> (abgerufen am 8. Juni 2026).
- [8] Microsoft, “Threats – Microsoft Threat Modeling Tool (STRIDE),” Microsoft Learn. [Online]. Verfügbar: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats> (abgerufen am 8. Juni 2026).
- [9] National Institute of Standards and Technology, “SP 800-38D: Galois/Counter Mode (GCM) and GMAC,” NIST CSRC. [Online]. Verfügbar: <https://csrc.nist.gov/pubs/sp/800/38/d/final> (abgerufen am 8. Juni 2026).
- [10] Internet Engineering Task Force, “RFC 6238: TOTP – Time-Based One-Time Password Algorithm.” [Online]. Verfügbar: <https://www.rfc-editor.org/rfc/rfc6238> (abgerufen am 8. Juni 2026).
- [11] Internet Engineering Task Force, “RFC 4226: HOTP – An HMAC-Based One-Time Password Algorithm.” [Online]. Verfügbar: <https://www.rfc-editor.org/rfc/rfc4226> (abgerufen am 8. Juni 2026).
- [12] World Wide Web Consortium (W3C), “Web Authentication: An API for Accessing Public Key Credentials Level 2,” W3C Recommendation. [Online]. Verfügbar: <https://www.w3.org/TR/webauthn-2/> (abgerufen am 8. Juni 2026).
- [13] Anthropic, “Claude Code (Sprachmodell Claude Opus 4.8),” KI-Assistent, unterstützend eingesetzt bei Konzeption, Implementierung und Ausarbeitung. [Online]. Verfügbar: <https://www.anthropic.com/claude-code> (verwendet im Juni 2026).